

MeshLab（个人 Fork）

BuildMeshLab passing

DOI 10.5281/zenodo.5114037

本仓库是 MeshLab 的个人 Fork（`ToraSpencer/meshlab`），基于官方仓库 cnr-isti-vclab/meshlab，用于学习其源码并进行个人定制。

- 个人开发分支：`develop_tora`
- 同步分支：`main`（仅用于跟随官方 `main`，不提交定制代码）
- 官方原始 README 保留在 README_ori.md，本文档为其中文翻译，并补充了本 Fork 的构建与上游同步说明。

本文目录

- 1. 项目简介
- 2. 发布版本
- 3. 构建与编译（Windows）
  - 3.1. 环境要求
  - 3.2. 安装 Qt（aqtinstall）
  - 3.3. 使用 Qt Creator 构建（推荐）
  - 3.4. 命令行构建（CMake 与 Ninja）
  - 3.5. 构建产物与运行
  - 3.6. 常见编译问题
- 4. 分支与 Pull Request
- 5. 从上游同步更新到开发分支
  - 5.1. 检查远程与分支配置
  - 5.2. 更新本地 main 与 Fork 的 main
  - 5.3. 合并上游更新到 develop_tora
  - 5.4. 处理合并冲突
  - 5.5. 常用检查命令
- 6. 仓库结构
- 7. 许可证
- 8. 版权
- 9. 引用
- 10. 联系方式
- 11. 反馈

1. 项目简介

MeshLab 是一个开源、可移植、可扩展的系统，用于处理和编辑非结构化的大型三维三角网格。它旨在帮助处理三维扫描中产生的、体积通常不算小的非结构化模型，提供了一整套用于编辑、清理、修复、检查、渲染和转换此类网格的工具。

MeshLab 主要基于开源 C++ 网格处理库 VCGlib，该库由 ISTI - CNR 的 Visual Computing Lab 开发。VCG 可以作为独立的大规模自动化网格处理流水线使用，而 MeshLab 则让交互式地试验这些算法变得容易。

MeshLab 支持 Windows、macOS 和 Linux。

2. 发布版本

你可以在 Releases 页面 找到适用于你所在平台的最新 MeshLab 版本。

你也可以通过下载最近一次 GitHub Actions 工作流的产物来测试最新的每夜构建（nightly）版本，下载方法参见这里。

3. 构建与编译（Windows）

上游提供了一套自动构建与部署脚本，位于 scripts 目录；这些脚本是 bash 编写的，在 Windows 上需要 Git Bash 或 WSL 等 bash 环境运行。日常开发更推荐使用 Qt Creator 或命令行 CMake。

3.1. 环境要求

组件	要求
操作系统	Windows 10/11，**仅支持 64 位**
编译器	Visual Studio 2019 或 2022，需安装「使用 C++ 的桌面开发」工作负载
Qt	Qt 5.15（仅支持 Qt 5，不支持 Qt 6；安装方式见 3.2 节）
CMake	3.18 或更高版本
Ninja	命令行构建推荐使用（官方脚本固定使用 Ninja）
Git	用于克隆与同步；子模块 `src/vcglib` 必须初始化

若尚未克隆仓库，请连同子模块一起克隆（本 Fork 的 `origin` 使用 SSH 地址）：

```
git clone --recursive git@github.com:ToraSpencer/meshlab.git
cd meshlab
```

若此前已克隆但未初始化子模块，执行：

```
git submodule update --init --recursive
```

> 说明：插件所需的第三方库在 CMake 配置阶段会自动下载到 `src/external/downloads`，首次配置需要联网。

3.2. 安装 Qt（aqtinstall）

MeshLab 需要 Qt 5.15 的 MSVC 2019 64-bit 开发库。Qt 官方没有为该 SDK 提供 winget 或 npm 包（winget 中的 `TheQtCompanyLtd.QtCreator` 只是 IDE，不含 Qt 库），推荐用 pip 安装 aqtinstall，从 Qt 官方源下载，可

指定版本、可脚本化、多版本并存，升级时只需改版本号重跑一条命令。

前置条件：Python 3.8+ 与 pip。安装并下载 Qt 5.15.2（以下命令在 PowerShell 中执行；`-O` 指定安装目录，位于用户目录下、不含盘符）：

```
pip install --upgrade aqtinstall
aqt install-qt windows desktop 5.15.2 win64_msvc2019_64 -O "$env:USERPROFILE\Qt"
```

安装结果为 `%USERPROFILE%\Qt\5.15.2\msvc2019_64`，其中包含 `bin\qmake.exe` 以及 `include\`、`lib\` 等目录。

> 关于编译器版本：Qt 5.15.2 官方只提供 MSVC 2019 64-bit 构建的库，没有 MSVC 2022 版本。MSVC 在 VS2015–2022 之间保持二进制兼容，直接用本机 VS2022 编译、链接即可（官方 CI 也是在 VS2022 环境中使用这批库构建的）。在 Qt Creator 中 Kit 会显示为 `MSVC2019 64bit`，编译器会自动识别为本机 VS2022。

常用操作：

目的	命令
查看 5.15.2 支持的架构	`aqt list-qt windows desktop --arch 5.15.2`
升级或切换版本	修改版本号后重跑 `aqt install-qt`，新旧版本目录并存
国内镜像加速	命令末尾追加 `-b https://mirrors.tuna.tsinghua.edu.cn/qt`

> Qt 5.15.2 是 5.15 系列最后一个提供开源二进制的版本；如需更高版本，须使用 Qt Online Installer 或自行从源码构建。

3.3. 使用 Qt Creator 构建（推荐）

1. 安装 Qt Creator IDE（与 Qt 库相互独立，可用 `winget install TheQtCompanyLtd.QtCreator`；当前 winget 源可能只提供预览版，介意时从官网下载稳定版）。
2. 在「工具 → 选项 → Kits → Qt Versions」中添加 `%USERPROFILE%\Qt\5.15.2\msvc2019_64\bin\qmake.exe`，并在「Kits」中为其选择 MSVC 64-bit 编译器。
3. 选择「文件 → 打开文件或项目」，打开仓库根目录的 `CMakeLists.txt`，Kit 选择上一步配置的 Qt 5.15.2 套件。
4. 在「构建设置」中选择构建类型（`Release` 或 `Debug`），构建目录建议放在源码树之外。
5. 点击「构建」。首次配置会下载外部依赖，耗时较长；完成后可在「运行」中选择 `meshlab` 目标启动程序。

3.4. 命令行构建（CMake 与 Ninja）

在「x64 Native Tools Command Prompt for VS 2022」中进入仓库根目录执行（普通 PowerShell 中 `cl.exe` 不在 `PATH`，无法直接构建）：

```
cd <仓库根目录>
cmake -S . -B build -G Ninja -DCMAKE_BUILD_TYPE=Release -
```

```
  -DCMAKE_PREFIX_PATH="$env:USERPROFILE\Qt\5.15.2\msvc2019_64"
  cmake --build build
```

命令中的 `build`、`install` 等均为相对仓库根目录的路径。`CMAKE_PREFIX_PATH` 需指向 Qt 安装目录中对应编译器的子目录；若用其他方式安装 Qt，替换为实际路径即可。

可选：安装到指定目录（分发前仍需补齐 Qt 运行时库，可用 `windeployqt` 或 `scripts/Windows` 下的部署脚本）：

```
cmake --install build --prefix install
```

常用 CMake 选项（在顶层 `CMakeLists.txt` 中定义）：

选项	默认值	作用
`MESHLAB_BUILD_WITH_DOUBLE_SCALAR`	`OFF`	使用 `double` 代替 `float` 作为标量类型
`MESHLAB_BUILD_MINI`	`OFF`	只构建核心部分，插件需手动指定目标
`MESHLAB_ENABLE_DEBUG_LOG_FILE`	`OFF`	将所有日志同时写入日志文件

3.5. 构建产物与运行

构建目录结构（以 `build` 为例）：

- `build/src/distrib/meshlab.exe`：主程序
- `build/src/distrib/plugins/`：插件
- `build/src/distrib/shaders/`：着色器
- `src/external/downloads/`：配置阶段自动下载的第三方库

命令行构建完成后可直接运行：

```
.\build\src\distrib\meshlab.exe
```

3.6. 常见编译问题

现象	处理方式
CMake 报错找不到 `Qt5`	检查是否安装了 Qt 5.15 的 MSVC 2019 64-bit 组件；命令行构建时用 `-DCMAKE_PREFIX_PATH="$env:USERPROFILE\Qt\5.15.2\msvc2019_64"` 指向实际安装目录
报错 `Minimum supported Qt5 version is 5.15!`	安装 Qt 5.15 或更高版本

现象	处理方式
编译找不到 vcglib 头文件	执行 `git submodule update --init --recursive`
提示 `cl.exe` 不是内部或外部命令	改用「x64 Native Tools Command Prompt」，或直接使用 Qt Creator
配置阶段下载外部库失败	检查网络与代理；已下载内容保留在 `src/external/downloads`，可重试配置

4. 分支与 Pull Request

上游仓库维护两个相关分支：

- `main`：存放**最新的 MeshLab 发布版本**，只包含缺陷修复、优化以及不影响已发布二进制的改动。发布后发现的问题会在此分支上制作「post-release」版本。
- `devel`：存放将在下一个 MeshLab 版本中出现的新功能。推送到 `main` 的缺陷修复会自动合并进 `devel`。

提交新改动或发起 Pull Request 之前，请选择适合改动的分支。如果只是缺陷修复，或是不修改软件行为的改动（例如文档笔误、部署系统调整等），请使用 `main`；如果是会改变软件行为或使用方式的新功能，请使用 `devel`。

本 Fork 的约定与之对应但相互独立：

- `main`：只接受上游 `main` 的快进更新，**不提交定制代码**。
- `develop_tora`：个人开发分支，承载全部定制改动；功能开发建议从它创建 `feature/*` 分支，完成后合回。

注意：上游还有一个 `devel` 分支（存放下一版本新功能），与本 Fork 的 `develop_tora` 是两个不同的分支，不要混淆。

5. 从上游同步更新到开发分支

建议每月检查一次，或在官方发布重要缺陷修复时同步。合并前先提交或暂存所有未完成的工作，以下命令均在仓库根目录执行。

5.1. 检查远程与分支配置

```
git remote -v
git branch -vv
git submodule status --recursive
```

正确配置应为：

- `origin` → `git@github.com:ToraSpencer/meshlab.git`（你的 Fork）
- `upstream` → `https://github.com/cnr-isti-vclab/meshlab.git`（官方仓库）

- 本地 `main` 追踪 `origin/main`，本地 `develop_tora` 追踪 `origin/develop_tora`

若尚未添加 `upstream`，执行：

```
git remote add upstream https://github.com/cnr-isti-vclab/meshlab.git
git fetch upstream
```

5.2. 更新本地 main 与 Fork 的 main

```
git status
git fetch upstream
git fetch origin
git switch main
git pull --ff-only origin main
git merge --ff-only upstream/main
git push origin main
```

`--ff-only` 可防止同步分支意外产生合并提交。若失败，不要直接 `git reset --hard` 或强推，先检查分叉原因。

5.3. 合并上游更新到 develop_tora

更稳妥的方式（推荐用于较大更新）：从最新 `develop_tora` 创建同步分支，验证后再合入。

```
git switch develop_tora
git pull --ff-only origin develop_tora
git switch -c sync/upstream-20261028
git merge main
git submodule update --init --recursive
# 此处执行编译、启动和回归测试
git push -u origin sync/upstream-20261028
```

随后在 GitHub 创建 PR：`sync/upstream-20261028` → `develop_tora`（分支名中的日期按实际同步日期替换）。

若更新较小且已确认无冲突，也可直接合并：

```
git switch develop_tora
git pull --ff-only origin develop_tora
git merge main
git submodule update --init --recursive
# 此处执行编译、启动和回归测试
git push origin develop_tora
```

同步后务必重新编译并做基本回归：启动程序、导入网格、运行典型滤镜。上游更新可能修改 `src/vcglib` 子模块指向，因此 `git submodule update --init --recursive` 不能省略。

5.4. 处理合并冲突

当 `git merge main` 报告冲突时：

```
git status
git diff --name-only --diff-filter=U
```

逐个打开冲突文件，理解官方改动与个人改动的意图，手动修改为最终期望的内容，不要盲目「全部接受当前」或「全部接受传入」。解决后：

```
git add <已解决的文件>
git merge --continue
```

若要撤销这次尚未完成的合并：

```
git merge --abort
```

子模块冲突时，先确认应使用哪个子模块提交，再进入子模块核查兼容性，最后在父仓库暂存更新后的 gitlink；不能仅凭冲突标记自动选版本。

5.5. 常用检查命令

```
git fetch upstream
git log --oneline main..upstream/main      # 官方 main 相比本地 main 的新提交
git log --oneline main..develop_tora       # 你的开发分支独有的提交
git diff --stat main...develop_tora        # 两条分支的差异摘要
git log --graph --oneline --decorate --all -30
git submodule status --recursive
```

6. 仓库结构

MeshLab 仓库组织如下：

- docs：用于生成 MeshLab 文档的 doxygen 脚本，详见其 README；
- `sample` 与 `textures`：用于测试的一组文件（网格、图像）；
- resources：软件本身以及部署系统生成最终 MeshLab 二进制时使用的文件；
- scripts：各平台的构建与部署脚本，详见其 README；
- src：MeshLab 及其插件的全部源码，详见其 README；
- `unsupported`：一组已不再包含在 MeshLab 中构建的旧插件。

`src` 内部结构：

- `src/cmake`：用于查找外部库的 CMake 脚本；
- `src/external`：下载 MeshLab 与部分插件所需外部库的 CMake 配置；
- `src/common`：MeshLab 及其插件使用的工具类与函数；
- `src/meshlab`：MeshLab 的 GUI 与核心；
- `src/meshlabplugins`：可添加到 MeshLab 的所有插件；
- `src/use_cpu_opengl`：仅在 Windows 下编译的工具，用于非 GPU 加速的 OpenGL 功能；
- `src/vcglib`：VCGlib 子模块。

7. 许可证

MeshLab 源码基于 GPL 许可证 发布。

8. 版权

```
    MeshLab
    https://www.meshlab.net
    All rights reserved.

    VCGLib  http://www.vcglib.net                                    o o
    Visual and Computer Graphics Library                           o     o
                                                                  _   O  _
    Paolo Cignoni                                                   \/)\/
    Visual Computing Lab  http://vcg.isti.cnr.it                   /\/|
    ISTI - Italian National Research Council                          |
    Copyright(C) 2005-2021                                            \
```

9. 引用



使用本工具时，请根据需要在官网 https://www.meshlab.net/#references 列出的参考文献中选择引用；如果不想花时间挑选，至少引用以下条目：

```
@software{meshlab,
  author       = {Cignoni, Paolo and Muntoni, Alessandro and Ranzuglia, Guido and
Callieri, Marco},
  title        = {MeshLab},
  publisher    = {Zenodo},
  doi          = {10.5281/zenodo.5114037}
}

@inproceedings {LocalChapterEvents:ItalChap:ItalianChapConf2008:129-136,
  booktitle = {Eurographics Italian Chapter Conference},
  editor    = {Vittorio Scarano and Rosario De Chiara and Ugo Erra},
  title     = {{MeshLab: an Open-Source Mesh Processing Tool}},
  author    = {Cignoni, Paolo and Callieri, Marco and Corsini, Massimiliano and
Dellepiane, Matteo and Ganovelli, Fabio and Ranzuglia, Guido},
```

```
  year      = {2008},
  publisher = {The Eurographics Association},
  ISBN      = {978-3-905673-68-5},
  DOI       = {10.2312/LocalChapterEvents/ItalChap/ItalianChapConf2008/129-136}
}
```

10. 联系方式

- Paolo Cignoni (paolo.cignoni (at) isti.cnr.it)
- Alessandro Muntoni (alessandro.muntoni (at) isti.cnr.it)

11. 反馈

对于可复现的缺陷、功能请求等，请使用上游仓库的 GitHub issues。

一般性使用问题请使用 StackOverflow。

与本 Fork 定制改动相关的问题，请提交到本 Fork 的 issues。